# Analysis Report



Share this Report

GC log file: **gc-2017_08_03-14_26.log.0**

Duration: **14 min 12 sec 703 ms**

## System Time greater than User Time

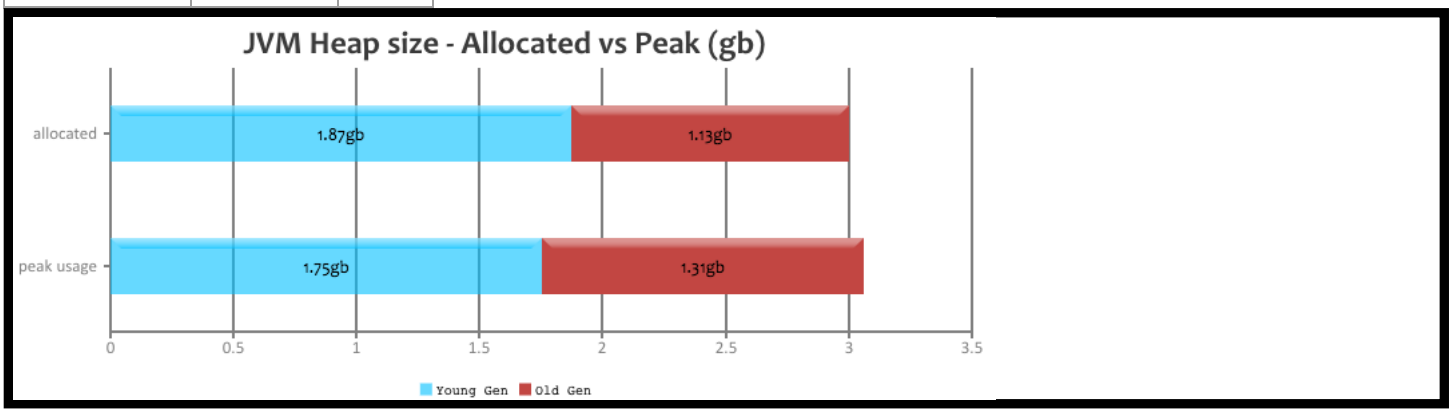
In **1 GC event(s)**, 'sys' time is greater than 'usr' time. It's not a healthy sign.

| Timestamp | User Time (secs) | Sys Time (secs) | Real Time (secs) |
|---|---|---|---|
| 2017-08-03T14:37:33 | 0.54 | 0.82 | 0.39 |

Read our recommendations to **reduce sys time (https://blog.gceasy.io/2016/12/11/sys-time-greater-than-user-time/)**

## JVM Heap Size

| Generation | Allocated | Peak |
|---|---|---|
| Young Generation | 1.87 gb | 1.75 gb |
| Old Generation | 1.13 gb | 1.31 gb |
| Total | 3 gb | 2.75 gb |




## Key Performance Indicators

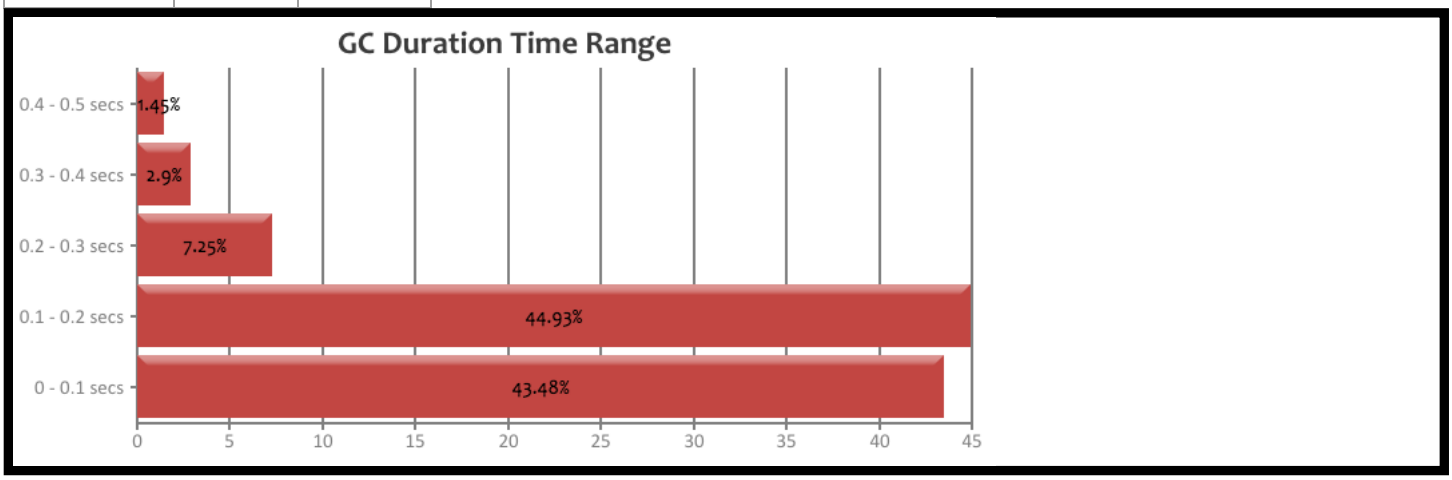
(Important section of the report. To learn more about KPIs, click here (https://blog.gceasy.io/2016/10/01/garbage-collection-kpi/))

1. **Throughput : 99.01%**
2. **Latency:**

| | |
|---|---|
| Avg GC Time | **103 ms** |
| Max GC Time | **460 ms** |

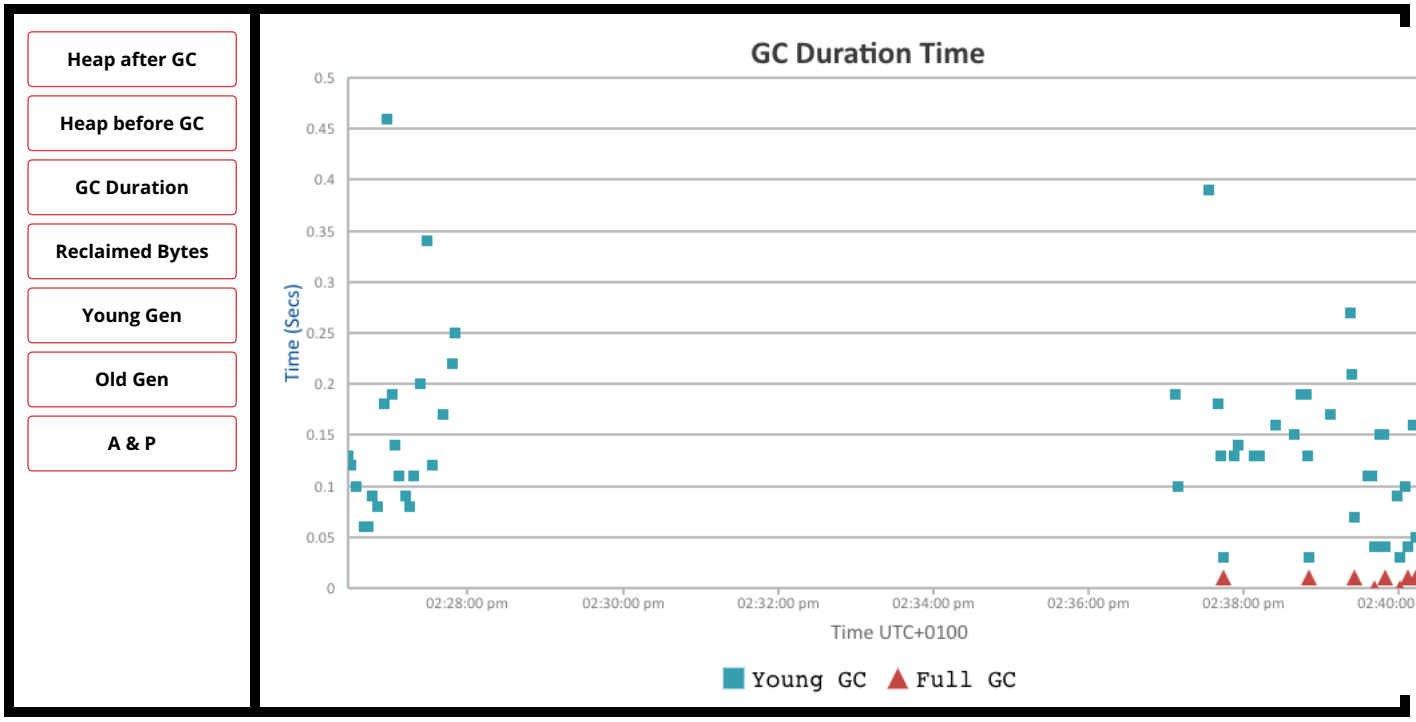
**GC Duration Time Range**:

| Duration (secs) | No. of GCs | Percentage |
|---|---|---|
| 0 - 0.1 | 30 | 43.478% |
| 0.1 - 0.2 | 31 | 88.406% |
| 0.2 - 0.3 | 5 | 95.652% |
| 0.3 - 0.4 | 2 | 98.551% |
| 0.4 - 0.5 | 1 | 100.0% |

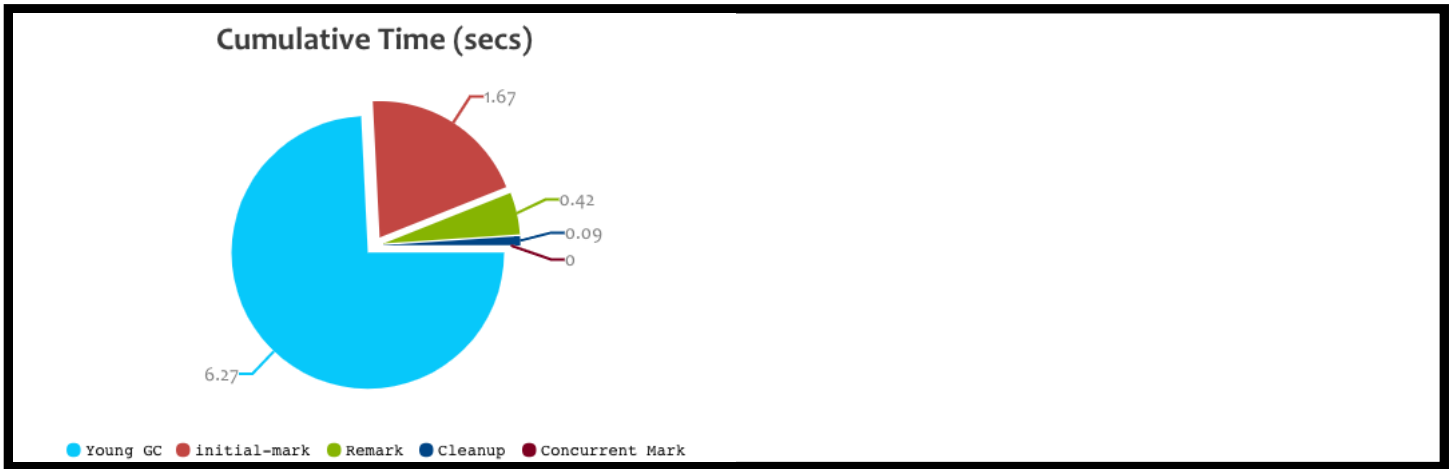


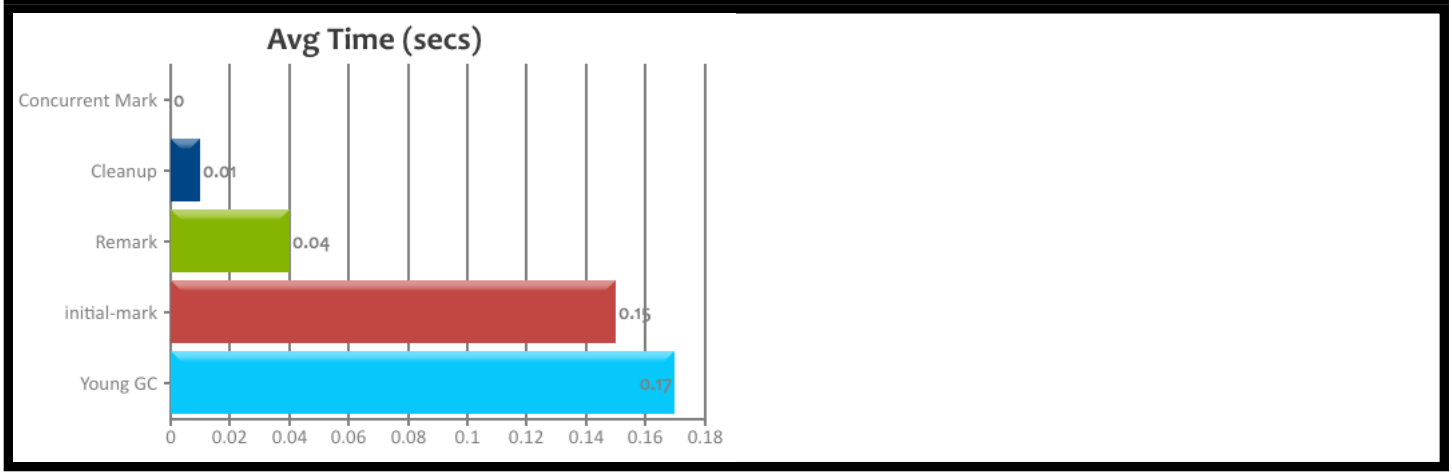

# Interactive Graphs

*(All graphs are zoomable)*



# G1 GC Statistics




| | Concurrent Mark | Cleanup | Young GC | Remark | initial-mark | Total |
|---|---|---|---|---|---|---|
| Count | 11 | 11 | 38 | 11 | 11 | 82 |
| Total GC Time | 0 | 86 ms | 6 sec 270 ms | 420 ms | 1 sec 670 ms | 8 sec 446 ms |
| Avg GC Time | 0 | 8 ms | 165 ms | 38 ms | 152 ms | 103 ms |
| Avg Time std dev | 0 | 4 ms | 84 ms | 12 ms | 43 ms | 94 ms |
| Min/Max Time | 0 / 0 | 0 / 10 ms | 0 / 460 ms | 0 / 70 ms | 0 / 220 ms | 0 / 460 ms |
| Avg Interval Time | 17 sec 585 ms | 17 sec 585 ms | 22 sec 703 ms | 17 sec 585 ms | 17 sec 589 ms | 20 sec 45 ms |

# Object Stats

(These are good metrics to include in your performance reports)

| | |
|---|---|
| Total created bytes | 45.03 gb |
| Total promoted bytes | 682 mb |
| Avg creation rate | 54.08 mb/sec |
| Avg promotion rate | 819 kb/sec |

## Memory Leak

No major memory leaks.

(**Note:** there are 8 flavours of OutOfMemoryErrors (https://tier1app.files.wordpress.com/2014/12/outofmemoryerror2.pdf). With GC Logs you can diagnose only 5 flavours of them(Java heap space, GC overhead limit exceeded, Requested array size exceeds VM limit, Permgen space, Metaspace). So in other words, your application could be still suffering from memory leaks, but need other tools to diagnose them, not just GC Logs.)

## Consecutive Full GC

None.

## Long Pause

None.

## Safe Point Duration

(To learn more about SafePoint duration, click here (https://blog.gceasy.io/2016/12/22/total-time-for-which-application-threads-were-stopped/))

Not Reported in the log.

## GC Causes

(What events caused the GCs, how much time it consumed?)

Not reported in the log.

## Tenuring Summary

Not reported in the log.

## Command Line Flags

Not reported in the log.

## Become a DevOps champion in your organization

(Best practises/tools)

- Use **fastThread.io** (https://fastthread.io/) to analyze thread dumps, core dumps and hs_err_pid files
- Do proactive Garbage Collection analysis on all your JVMs (not just one or two) using the GC log analysis API (https://blog.gceasy.io/2016/06/18/garbage-collection-log-analysis-api/)